

Exercise Sheet 3

Test yourself

Exercise 3.1 LL(k)

A grammar is an $LL(k)$ -grammar for some $k \in \mathbb{N}$ if whenever there exist $u, x, y \in V_T^*$ with $k : x = k : y, Y \in V_N$ and $\alpha, \beta, \gamma \in (V_T \cup V_N)^*$ such that

$$\begin{array}{ccccccc}
 S & \xrightarrow[lm]{*} & uY\alpha & \xRightarrow[lm]{} & u\beta\alpha & \xrightarrow[lm]{*} & ux \\
 S & \xrightarrow[lm]{*} & uY\alpha & \xRightarrow[lm]{} & u\gamma\alpha & \xrightarrow[lm]{*} & uy
 \end{array}$$

then $\beta = \gamma$.

A language L is an $LL(k)$ -language if there exists an $LL(k)$ -grammar that generates L .

1. Prove that for each $k \in \mathbb{N}$ there exists a grammar which is $LL(k+1)$ but not $LL(k)$.
2. Prove that for each $k \in \mathbb{N}$ an $LL(k)$ -grammar is an $LL(k+1)$ -grammar.
3. Investigate $LL(0)$ -languages. How many words can an $LL(0)$ -language contain?
4. A grammar is left-recursive if it has a production of the form $A \rightarrow A\mu$. Show that a left-recursive grammar is not $LL(k)$ for any k .

Exercise 3.2 Parsetrees

Consider two grammars that you should already know from the lecture:

$$\begin{array}{lcl}
 G_0: & E & \rightarrow E + T \mid T \\
 & T & \rightarrow T * F \mid F \\
 & F & \rightarrow (E) \mid \text{id}
 \end{array}$$

$$G_1: \quad E \rightarrow E + E \mid E * E \mid (E) \mid \text{id}$$

Draw *any* parse tree for the word **id * id + id**.

Exercise 3.3 Expression Grammar

Consider the following ambiguous grammar:

$$E \rightarrow E \ o_1 \ E$$

$$E \rightarrow E \ o_2 \ E$$

$$E \rightarrow E \ o_3 \ E$$

$$E \rightarrow \mathbf{id} \mid (E)$$

1. Disambiguate the grammar by encoding the following operator precedence and associativity rules into the new grammar:

- o_1 binds stronger than o_2
- o_2 binds stronger than o_3
- o_1 and o_3 are left-associative
- o_2 is right-associative

Examples ($x, y, z \in \mathbf{id}$):

$$x \ o_2 \ y \ o_1 \ z = x \ o_2 \ (y \ o_1 \ z)$$

$$x \ o_3 \ y \ o_2 \ z = x \ o_3 \ (y \ o_2 \ z)$$

$$x \ o_1 \ y \ o_1 \ z = (x \ o_1 \ y) \ o_1 \ z$$

$$x \ o_2 \ y \ o_2 \ z = x \ o_2 \ (y \ o_2 \ z)$$

$$x \ o_3 \ y \ o_3 \ z = (x \ o_3 \ y) \ o_3 \ z$$

2. Eliminate any left recursion in your grammar.
3. Compute the $First_1$ and $Follow_1$ sets.